PRODUCT CREATION BRIEF

Employee Requests & Time Tracking System

Full-Stack Product Challenge · From branding to production

Mission Build a complete employee operations platform that combines internal employee requests with time and attendance tracking, supported by Employee, Manager, and HR roles.

Backend	Django + Django REST Framework
Database	PostgreSQL
Development Process	SpecKit-guided engineering using suitable AI development tools
Final Outcome	Tested, secure, responsive, and deployed production application

1. Project Description

Build a complete employee operations platform from scratch that combines internal employee requests with time and attendance tracking. Each team must take the product from idea to production: define the product identity, design the user experience, specify the solution using SpecKit, implement the frontend and backend, verify quality, and deploy a working application.

The product should replace fragmented request and time-reporting processes with one secure workspace. Employees can submit and follow requests and record their working time. Managers can review team activity and approvals. HR has organization-wide oversight, employee administration, reporting, and system visibility.

2. Product Objectives

- Centralize employee requests and make ownership and status visible.
- Provide a clear, reliable workflow for recording, submitting, and reviewing working hours.
- Enforce role-based permissions in both the user interface and Django backend.
- Deliver a consistent product brand and responsive experience across desktop, tablet, and mobile.
- Demonstrate a structured SpecKit and AI-assisted workflow from requirements through deployment.

3. Required Product Journey

Discover	Name	Brand	Specify	Build	Deploy

Problem understanding → product naming → branding → product requirements → UX/UI → SpecKit constitution → feature specifications → technical planning → development → testing → deployment → final demo.

4. Users and Roles

Role	Primary Goal	Core Access
Employee	Manage personal requests and working time	Own profile, own requests, own timesheets, status and activity history
Manager	Review and manage the direct team	Team requests, approvals, timesheet review, team hours and dashboards
HR	Oversee workforce operations	Employees, all requests, timesheets, reports, configuration and system-level visibility

5. Role Capabilities

Employee

- Sign in securely and manage a personal profile.
- Create internal requests and attach supporting information.
- View only permitted requests, track status, and review activity history.
- Create, edit, and submit timesheet entries.
- Review personal working hours and timesheet approval status.

Manager

- Access employees who report directly to the manager.
- Review, approve, reject, or return relevant employee requests.
- Review submitted timesheets and return entries for correction.
- Monitor team working hours and request activity.
- Use team-level dashboards, search, filters, and reporting views.

HR

- Manage employee records, reporting lines, roles, and account status.
- Oversee requests and timesheets across the organization.
- Review reports, trends, pending work, and approval status.
- Configure request types and organization-level settings.
- Access system-wide operational visibility according to policy.

6. Core Product Modules

Module	Expected Capability
Authentication & Users	Secure access, profiles, employee records and reporting lines.
Role-Based Access	Employee, Manager and HR authorization enforced server-side.
Employee Requests	Create, view, update, submit and track internal requests.
Request Approvals	Manager and HR review where applicable, with clear status history.
Timesheets	Create daily entries, record from/to times or hours, edit drafts and submit.
Timesheet Review	Manager approval, rejection or return for correction.
Dashboards	Relevant individual, team and organization-level summaries.
Search & Filters	Find requests and timesheets using useful filters and sorting.
Notifications	In-app feedback for submission, approval, rejection and returned work.
Reporting	Working-hour and request summaries suitable for HR and managers.

7. Key Workflows

Employee Request Flow

Create	Submit	Manager	HR if needed	Decision

Typical request types may include leave, equipment, HR service, work-from-home, or a general internal request.

Timesheet Flow

Log time	Review	Submit	Manager review	Final status

A submitted timesheet should become read-only unless it is returned for correction or reopened by an authorized role.

8. Branding Deliverables

Each team creates a concise, coherent identity that is suitable for an employee operations product.

- Product name, meaning, naming rationale, and tagline.
- Primary logo, product icon, light version, and dark version.
- Color palette covering primary, secondary, success, warning, error, and neutral colors.
- English and Arabic typography choices where bilingual support is included.
- Three to five brand-personality attributes.
- Basic usage rules for color, spacing, imagery, corner radius, and emphasis.

9. UX and UI Requirements

- Define employee, manager, and HR user flows before implementation.
- Create a small, reusable design system with buttons, inputs, dropdowns, tables, cards, badges, dialogs, toasts, and navigation.
- Support desktop, tablet, and mobile layouts with consistent hierarchy and interaction behavior.
- Provide loading, empty, error, validation, disabled, and success states for asynchronous screens.
- Use accessible labels, keyboard-friendly interactions, readable contrast, and visible focus states.
- Keep request submission and time logging direct, understandable, and low-friction.

10. Technical Requirements

Backend	Django with Django REST Framework
Database	PostgreSQL with versioned schema migrations
Frontend	Team-selected modern framework; responsive and production-ready
API	Documented REST endpoints with validation, authorization, filtering and pagination
Security	Server-side RBAC, ownership checks, secure secrets, input validation and safe error responses
Deployment	Publicly reachable frontend, backend and database environment; not localhost-only

11. Suggested Architecture



Optional services such as object storage, email, background jobs, Redis, or application monitoring may be introduced only when they serve a documented requirement.

12. Quality and Security Standards

- Authorization must be enforced in Django, not only by hiding frontend controls.
- Users must not access or change resources outside their permitted scope.
- Every input must be validated and return useful, safe error responses.
- Secrets and credentials must never be committed to source control.
- Database changes must use migrations; destructive resets are not an acceptable delivery method.
- Critical workflows must include automated tests.
- Large result sets should use pagination and efficient Django query patterns.

13. SpecKit Requirements

Before implementation, each team must define a project constitution that governs product, UX, design, backend, database, security, and quality decisions. Specifications should translate requirements into testable behavior and remain the source of truth throughout development.

Suggested Repository Structure

- /specs/constitution.md
- /specs/001-brand-product-foundation/spec.md, plan.md, tasks.md
- /specs/002-authentication-rbac/spec.md, plan.md, tasks.md
- /specs/003-employee-requests/spec.md, plan.md, tasks.md
- /specs/004-timesheets/spec.md, plan.md, tasks.md
- /specs/005-dashboard-reporting/spec.md, plan.md, tasks.md

Constitution Principles

Area	Required Principle
Product	Every feature solves a documented user problem and stays within the agreed MVP.
UX	Every asynchronous screen provides loading, empty, error, and success behavior.
Design	Use the defined design system; avoid arbitrary spacing, color, or component variants.
Backend	Permissions and business rules are authoritative on the server.
Database	All schema changes use migrations and preserve existing data.
Security	Validate inputs, protect ownership boundaries, and keep secrets outside the repository.
Quality	Critical workflows and authorization rules require automated tests.

14. AI-Assisted Development

Teams may select and use any AI tools that support their needs during product discovery, branding, UX/UI design, specification, coding, testing, documentation, or deployment. Examples include ChatGPT, Codex, Claude, Gemini, GitHub Copilot, Cursor, Devin AI, and other appropriate tools. Teams may combine multiple tools when useful. The objective is to demonstrate disciplined AI-assisted software engineering through clear context, specification-led implementation, human review, and evidence-based verification—not unreviewed AI output.

1. Prepare the SpecKit constitution and relevant feature specification.
2. Create or update the technical plan and executable task list.
3. Select the AI tool best suited to the current task and provide clear context, constraints, acceptance criteria, and relevant repository information.
4. Review all AI-generated designs, content, plans, code, and configuration for architecture, security, UX, correctness, and scope compliance.
5. Run automated tests and complete manual verification of the affected flow.

6. Commit the verified change with clear traceability to the specification and task.

AI Usage Evidence

- Which AI tools were used and why each tool was selected.
- What each AI tool generated, suggested, or changed.
- What decisions were made by the team.
- What the team reviewed, rejected, or corrected after generation.
- How the result was tested and checked against the constitution.

15. Minimum Testing Scope

- Authentication, sign-in failure, and protected-route behavior.
- Employee, Manager, and HR permissions and ownership boundaries.
- Employee request creation, validation, submission, review, and final status.
- Timesheet creation, editing, submission, review, and return for correction.
- Unauthorized API access and attempts to access another manager's team.
- Search, filters, pagination, and empty result behavior.
- Responsive employee flows on representative mobile and desktop sizes.
- Error handling for expected backend and network failures.

16. Deployment Requirements

The final application must be deployed and accessible through a working URL. A localhost-only submission is incomplete.

- Deploy the frontend and Django API to suitable production environments.
- Use a managed or properly secured PostgreSQL database.
- Configure environment variables and secrets outside source control.
- Apply database migrations as part of the deployment process.
- Provide demo credentials for Employee, Manager, and HR roles.
- Include a short README with setup, architecture, test, and deployment instructions.

17. Final Deliverables

Deliverable	Minimum Content
Brand Package	Product name, tagline, logo variants, icon, palette, typography and basic rules
Product Documentation	Vision, objectives, personas or role definitions, scope and user flows
Design	Design system and responsive main screens
SpecKit	Constitution, feature specifications, plans and task files
Source Code	Frontend, Django backend, migrations and automated tests
Technical Documentation	Architecture, database model and API documentation
Production Delivery	Live URL, repository, demo credentials and deployment notes
Presentation	Product story, brand, demo, architecture, SpecKit and AI development process

18. Competition Period and Stage Schedule

The project begins on 8 September 2026 and concludes with final judging on 5 October 2026. Each stage has a defined outcome and review checkpoint so teams receive structured evaluation throughout the product journey.

Competition Period Start: 8 September 2026 | Final Submission and Judging: 5 October 2026 | Total Duration: 28 calendar days

Stage	Dates	Focus	Required Outcome
1	8–9 Sep	Discovery & Scope	Problem definition, users, goals and MVP scope
2	10–11 Sep	Naming & Branding	Name, tagline, logo direction, palette and typography
3	12–14 Sep	Product Definition	Requirements, roles, permissions and user stories
4	15–18 Sep	UX & UI Design	Flows, wireframes, design system and responsive screens
5	19–21 Sep	SpecKit	Constitution, feature specifications, plans and tasks
6	22–23 Sep	Technical Design	Architecture, database, API and security design
7	24–29 Sep	Implementation	Functional Django backend and responsive frontend
8	30 Sep	Roles & Security Review	Verified RBAC, ownership and protected API behavior
9	1–2 Oct	Testing & Quality	Automated tests, manual QA and defect resolution
10	3 Oct	Deployment & Documentation	Live system, migrations, README and credentials
11	4 Oct	Feature Freeze & Rehearsal	Release candidate, evidence pack and demo rehearsal
12	5 Oct	Final Demo & Judging	Live presentation, product demonstration and Q&A

19. Judging Principles

Judges should evaluate the quality of the product decisions and implementation—not the number of features. A smaller, coherent, secure and tested product should score higher than a larger but incomplete solution.

- Judge only evidence available in the repository, deployed application, documentation and live demonstration.
- Apply the same scoring definitions and maximum points to every team.
- Evaluate frontend appearance together with behavior, accessibility, responsiveness and state handling.
- Verify permissions through backend/API behavior; hidden frontend controls alone do not prove authorization.
- Reward clear traceability from requirement to specification, task, implementation and test.
- Evaluate AI usage by the quality of team direction, review and verification—not by the chosen AI product.
- Record a short justification for every score to support transparent final feedback.

20. Scoring Summary

Stage	Evaluation Area	Points
1	Problem Understanding & MVP Scope	**7**
2	Product Naming & Branding	**8**
3	Product Requirements & Role Definition	**7**
4	UX, UI & Design System	**10**
5	SpecKit Usage & Traceability	**12**
6	Architecture, Database & API Design	**10**
7	Django Backend Quality	**12**
8	Frontend Quality & Responsiveness	**12**
9	Roles, Authorization & Security	**8**
10	Testing & Quality Assurance	**6**
11	Deployment & Documentation	**4**
12	Final Demo & Technical Presentation	**4**
	TOTAL	**100**

21. Rating Scale

Rating	Definition

Rating	Definition
5 — Excellent	Complete, coherent and production-ready; evidence exceeds the requirement.
4 — Strong	Requirement is fully met with only minor weaknesses.
3 — Acceptable	Core requirement works, but quality, depth or consistency is incomplete.
2 — Weak	Partially implemented; important gaps or defects remain.
1 — Minimal	Very limited evidence; requirement is mostly unmet.
0 — Missing	No usable evidence, implementation or demonstration.

Scoring method: judges select a rating from 0 to 5 for each criterion. The awarded points are calculated proportionally against that criterion's maximum score. Final team score equals the average of all judges' totals out of 100.

22. Detailed Evaluation by Stage

The following rubric defines what judges must inspect at each stage and how the available points should be awarded.

Stage	Max	Judging Criteria	Required Evidence
1. Discovery & Scope	**7**	Problem clarity (2); target users and needs (2); measurable goals (1); realistic MVP and exclusions (2).	Product brief, personas, MVP statement
2. Naming & Branding	**8**	Name relevance and memorability (2); tagline and positioning (1); logo/icon system (2); colors and typography (2); consistency and accessibility (1).	Brand rationale, logo variants, palette, type rules
3. Product Definition	**7**	Clear functional requirements (2); Employee/Manager/HR responsibilities (2); workflows and acceptance criteria (2); scope consistency (1).	PRD, role matrix, user stories, flows
4. UX & UI	**10**	Flow clarity (2); design-system consistency (2); responsive behavior (2); loading/empty/error/success states (2); accessibility and usability (2).	Design files or coded screens, responsive evidence
5. SpecKit	**12**	Useful constitution (2); testable specifications (3); realistic technical plans (2); actionable tasks (2); traceability (2); specification maintenance (1).	Constitution, specs, plans, tasks, commit history
6. Technical Design	**10**	Architecture boundaries (2); database model and migrations (2); API contracts and validation (2); security design (2); maintainability and justified choices (2).	Architecture diagram, ERD, API documentation
7. Django Backend	**12**	Correct domain behavior (3); code structure and maintainability (2); API quality (2); data integrity and migrations (2); query efficiency (1); error handling (1); documentation (1).	Running API, source code, migrations and API tests
8. Frontend	**12**	Functional core flows (3); component quality (2); responsive layouts (2); state management (1); API integration (2); validation and feedback (1); accessibility (1).	Deployed UI, source code and browser evidence
9. Roles & Security	**8**	Server-side RBAC (3); ownership and reporting-line restrictions (2); authentication/session safety (1); input and secret handling (1); unauthorized-access tests (1).	Permission tests, API responses, configuration review

Stage	Max	Judging Criteria	Required Evidence
10. Testing & QA	**6**	Critical automated tests (2); authorization tests (1); manual test coverage (1); defect handling (1); stable main workflows (1).	Test suite, test report, QA checklist
11. Deployment & Docs	**4**	Working live environment (2); safe configuration and migrations (1); usable README and demo credentials (1).	Live URL, deployment record, README
12. Final Demo	**4**	Clear product story (1); successful live demonstration (1); technical and SpecKit explanation (1); concise answers and team ownership (1).	Presentation, live demo and Q&A

23. Stage Review and Feedback Process

Step	Review Action
1	At the end of each scheduled stage, the team submits the required evidence or presents it during a short checkpoint.
2	Judges record a provisional rating, the strongest aspect, the most important gap, and one recommended next action.
3	A provisional stage score may be improved before feature freeze when the team submits corrected evidence.
4	After feature freeze on 4 October 2026, only critical deployment or presentation fixes should be accepted.
5	The final score is confirmed after the live demo and repository review on 5 October 2026.

24. Mandatory Rules and Score Adjustments

Issue	Scoring Effect
Secret, password or private key committed to the repository	−10 points and immediate security review
Authorization implemented only in the frontend	Roles & Security score capped at 2/8
No working deployed application	Deployment score is 0/4
Database schema changed by destructive reset instead of migration	−5 points
AI-generated work submitted without team review or disclosure	AI evidence marked incomplete; affected criterion may receive 0
Core demo data or behavior is fabricated	Affected stage receives 0 and judges document the finding

25. Tie-Breaking Order

Priority	Tie-Breaking Rule
1	Higher Roles, Authorization & Security score.
2	Higher SpecKit Usage & Traceability score.
3	Higher combined Backend and Frontend score.
4	Higher Testing & Quality Assurance score.
5	Final decision by the judging panel based on demonstrated product coherence.